INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

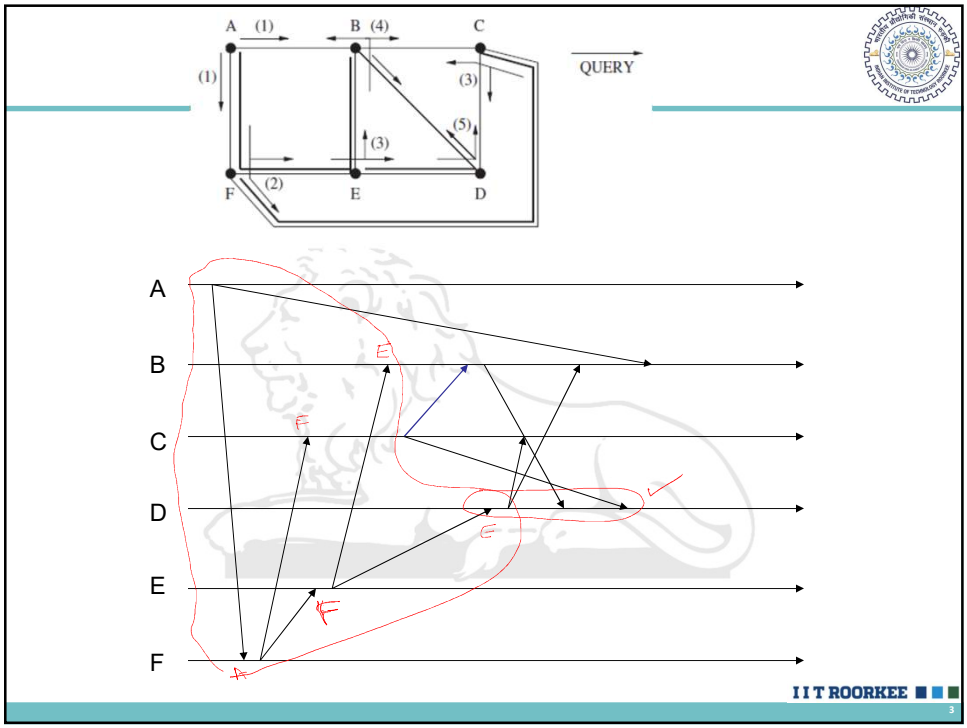
Distributed Systems

Elementary Graph Algorithms

Asynchronous single-initiator spanning tree algorithm using flooding

```
(local variables)
int parent ⟵ ⊥
set of int Children, Unrelated ⟵ ∅
set of int Neighbors ⟵ set of neighbors
(message types)
QUERY, ACCEPT, REJECT

(1) When the predesignated root node wants to initiate the algorithm:
(1a) if (i = root and parent = ⊥) then
(1b)     send QUERY to all neighbors;
(1c)     parent ⟵ i.

(2) When QUERY arrives from j:
(2a) if parent = ⊥ then
(2b)     parent ⟵ j;
(2c)     send ACCEPT to j;
(2d)     send QUERY to all neighbors except j;
(2e)     if (Children ∪ Unrelated) = (Neighbors \ {parent}) then
(2f)         terminate.
(2g) else send REJECT to j.
```

IIT ROORKEE



Leader election

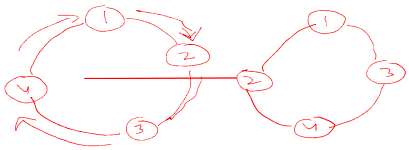
- Algorithms such as the minimum spanning tree and broadcast / convergecast require a leader process that starts computation
- A leader is required in many distributed systems because algorithms are typically not completely symmetrical, and some process has to take the lead in initiating the algorithm
- Another reason is that we would not want all the processes to replicate the algorithm initiation, to save on resources.
- Leader election requires that all the processes agree on a common distinguished process (*leader)*

IIT ROORKEE

Leader election

- An algorithm for leader election may vary in the following aspects:
  - **Communication mechanism**: the processors are either synchronous or asynchronous
  - **Process names**: whether processes have a unique identity or are indistinguishable (anonymous).
  - **Network topology**: for instance, ring, acyclic graph or complete graph
  - **Size of the network**: the algorithm may or may not use knowledge of the number of processes in the system.

Leader election in Ring



- Assume a ring topology
- It is easy to design and analyze the algorithm in this particular setting
- lower bounds and impossibility results for the ring topology also applies to an arbitrary topologies
- Each process has a left neighbor and a right neighbor

Leader election

Leader election in rings

Synchronous rings: $O(n)$ where size of the ring is n

Asynchronous rings: $O(n^2), O(n \log n)$

Anonymous rings: No deterministic algorithms

Leader election

Lelang, Chang, and Roberts (LCR) algorithm:

Assumptions:

- Processes are connected by an asynchronous unidirectional ring.
- All processes have unique identifiers.
- Idea:
  - Processes circulate their IDs; highest ID wins
- Worst case msg complexity n . (n + 1) / 2; $O(n^2)$
- time complexity O(n)

Lelang, Chang, and Roberts (LCR) algorithm

Methodology:

- Each process in the ring sends its identifier to its left neighbor.
- When a process Pi receives the identifier k from its right neighbor Pj , it acts as follows:
- i < k: forward the identifier k to its left neighbor;
- i > k: ignore the message received from neighbor j;
- i = k: Pi can declare itself the leader.
- Pi can then send another message around the ring

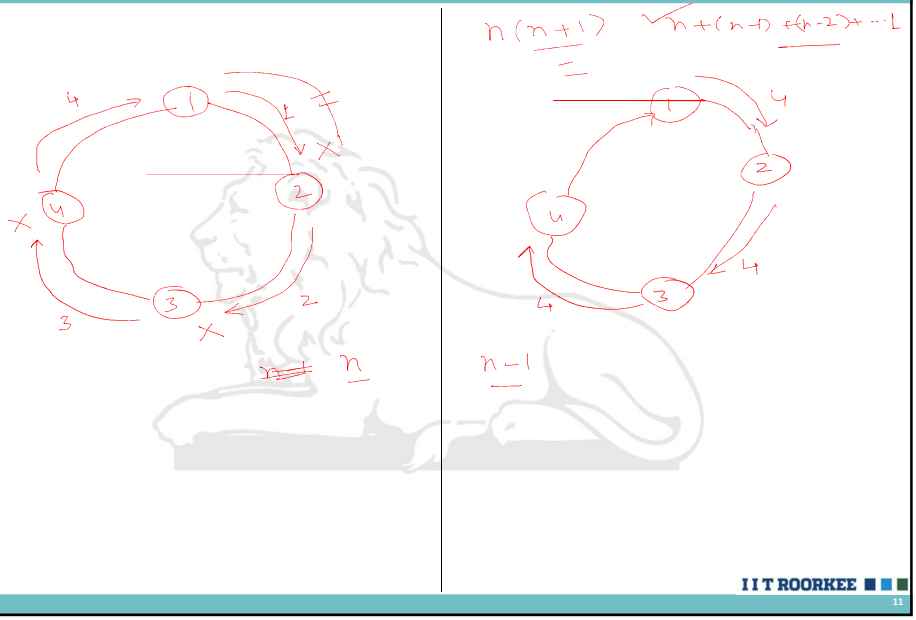
Lelang, Chang, and Roberts (LCR) algorithm

```
(variables)
boolean participate ← false          // becomes true when P_i is included in the MIS
(message types)
PROBE integer                        // contains a node identifier
SELECTED integer                     // announcing the result

(1) When a process wakes up to participate in leader election:
(1a) send PROBE(i) to right neighbor;
(1b) participate ⟵ true.

(2) When a PROBE(k) message arrives from the left neighbor P_j:
(2a) if participate = false then execute step (1) first.
(2b) if i > k then
(2c)         discard the probe;
(2d) else if i < k then
(2e)         forward PROBE(k) to right neighbor;
(2f) else if i = k then
(2g)         declare i is the leader;
(2h)         circulate SELECTED(i) to right neighbor;

(3) When a SELECTED(x) message arrives from left neighbor:
(3a) if x ≠ i then
(3b)         note x as the leader and forward message to right neighbor;
(3c) else do not forward the SELECTED message.
```

Message Complexity

Time Complexity

Example (LCR Algorithm)



Hirschberg and Sinclair Algorithm

- The $O(n^2)$ message cost can be reduced to $O(n \log n)$ by using a binary search in both directions
- Requires bidirectional ring
- Processors can detect from which direction a received message originated
- But "left" may not mean the same to all processors
- In round k, the token is circulated to 2^k neighbors on both the left and right sides.
- To cover the entire ring, a logarithmic number of steps are needed.

Hirschberg and Sinclair Algorithm

- Sendboth: sends messages in both directions.
- Sendpass: pass a (possibly modified) message in a circular manner.
- Sendecho: send a responsive message back in the direction from which message has been received

Hirschberg and Sinclair Algorithm

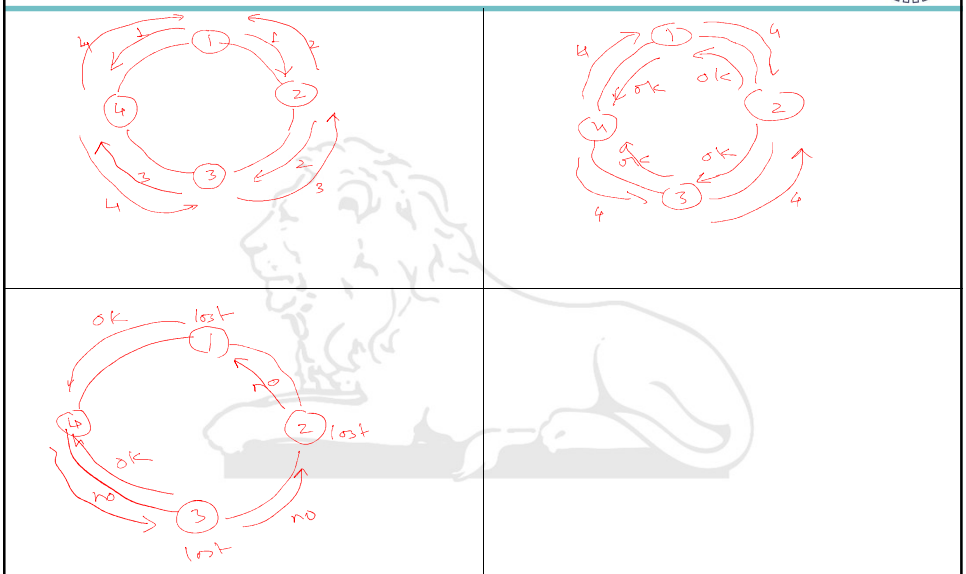
The Algorithm

To run for election:
    status ← "candidate"
    maxnum ← 1
    WHILE status = "candidate" DO
        sendboth ("from", myvalue, 0, maxnum)
        await both replies (but react to other messages)
    IF either reply is "no" THEN status ← "lost"
        maxnum ← 2*maxnum
    OD

On receiving message ("from", value, num, maxnum):
    IF value < myvalue THEN *sendecho* ("no", value)
    IF value > myvalue THEN DO
        status ← "lost"
        num ← num + 1
        IF num < maxnum THEN *sendpass* ("from", value, num, maxnum)
            ELSE *sendecho* ("ok", value)
    OD
    IF value = myvalue THEN status ← "won"

On receiving message ("no", value) or ("ok", value)
    IF value ≠ myvalue THEN *sendpass* the message
        ELSE this is a reply the processor was awaiting

Hirschberg and Sinclair Algorithm (Example)



Hirschberg and Sinclair Algorithm

- Consider that in each round, a process tries to become a leader, and only the winners in round k can proceed to round k+1.
- A process i is a leader in round k if and only if i is the highest identifier among 2^k neighbors in both directions.
- Hence, any pair of leaders after round k are at least 2^k apart.
- Hence the number of leaders diminishes logarithmically as $n/2^k$
- Observe that in each round, there are at most O(n) messages sent, using the suppression technique of the LCR algorithm.
- Thus the overall complexity is $O(n \cdot \log n)$.

Fault Tolerance & Recovery

Classification of faults:

- **based on component that failed**
  - program/process
  - processor/machine
  - link
  - storage
  - clock
- **based on behavior of faulty component**
  - Crash – just halts
  - Fail-stop – crash with additional conditions
  - Omission – fails to perform some steps / messages lost
  - Timing – violates timing constraints
  - Byzantine – behaves arbitrarily

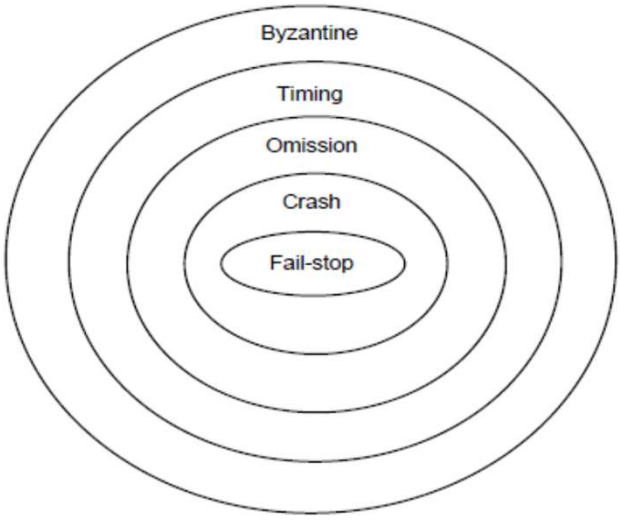


Figure 2: Failure model hierarchy

By Shivkaant Mishra and Richard D Schlichting

Types of tolerance:

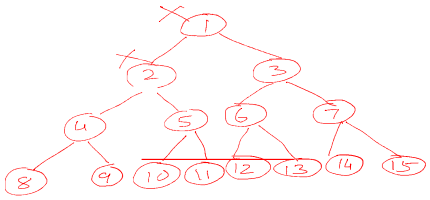
TMR

- Masking – system always behaves as per specifications even in presence of faults
- Non-masking – system may violate specifications in presence of faults. Should at least behave in a well-defined manner

Fault tolerant system should specify:

- Class of faults tolerated
- what tolerance is given from each class

show all quorums if node 1 and 2 are down (N = 15)



{3, 6, 12, 4, 8, 5, 10}, {3, 6, 12, 4, 9, 5, 10}
{3, 6, 12, 4, 8, 5, 11}, {3, 6, 12, 4, 9, 5, 11}
{3, 6, 13, 4, 8, 5, 10}, {3, 6, 13, 4, 9, 5, 10}
{3, 6, 13, 4, 8, 5, 11}, {3, 6, 13, 4, 9, 5, 11}

{3, 7, 14, 4, 8, 5, 10}, {3, 7, 15, 4, 9, 5, 10}
{3, 7, 14, 4, 8, 5, 11}, {3, 7, 15, 4, 9, 5, 11}
{3, 7, 14, 4, 8, 5, 10}, {3, 7, 15, 4, 9, 5, 10}
{3, 7, 14, 4, 8, 5, 11}, {3, 7, 15, 4, 9, 5, 11}

Queue of all sites after 4

A: | self

B: | self | A | C

C: | D | self

D: | self

E: nil

F: | B

G: nil

Holder variables and Queues at all sites after B gets the token

$Holder_B$ = self

$Holder_F$ = B

others same

B: | A | C after self is removed

F: nil